

C++ Standard Library Issues Resolved Directly In Kona

Doc. no. P0625R0

Date: Revised 2017-03-03 at 22:03:08 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

[2788](#). `basic_string` range mutators unintentionally require a default constructible allocator

Section: 21.3.2.6.2 [string.append], 21.3.2.6.3 [string.assign], 21.3.2.6.4 [string.insert], 21.3.2.6.6 [string.replace] **Status:** [Immediate](#) **Submitter:** Billy O'Neal III **Opened:** 2016-10-25 **Last modified:** 2017-03-03

Priority: 2

View all issues with [Immediate](#) status.

Discussion:

Email discussion occurred on the [lib reflector](#).

`basic_string`'s mutation functions show construction of temporary `basic_string` instances, without passing an allocator parameter. This says that `basic_string` needs to use a default-initialized allocator, which is clearly unintentional. The temporary needs to use the same allocator the current `basic_string` instance uses, if an implementation needs to create a temporary at all.

libc++ already does this; I believe libstdc++ does as well (due to the bug report we got from a user that brought this to our attention), but have not verified there. I implemented this in MSVC++'s STL and this change is scheduled to ship in VS "15" RTW.

[2016-11-12, Issaquah]

Sat AM: Priority 2

Alisdair to investigate and (possibly) provide an alternate P/R

[2017-02-13 Alisdair responds:]

Looks good to me - no suggested alternative

[Kona 2017-02-28]

Accepted as Immediate.

Proposed resolution:

This wording is relative to N4606.

1. In 21.3.2.6.2 [string.append], add the allocator parameter to the range overload temporary:

```
template<class InputIterator>
basic_string& append(InputIterator first, InputIterator last);
```

-19- *Requires:* [first, last) is a valid range.

-20- *Effects:* Equivalent to `append(basic_string(first, last, get\_allocator\(\)))`.

-21- *Returns:* *this.

2. In 21.3.2.6.3 [string.assign], add the allocator parameter to the range overload temporary:

```
template<class InputIterator>
basic_string& assign(InputIterator first, InputIterator last);
```

-23- *Effects:* Equivalent to `assign(basic_string(first, last, get\_allocator\(\)))`.

-24- *Returns:* *this.

3. In 21.3.2.6.4 [string.insert], add the allocator parameter to the range overload temporary:

```
template<class InputIterator>
iterator insert(const_iterator p, InputIterator first, InputIterator last);
```

-23- *Requires:* p is a valid iterator on *this. [first, last) is a valid range.

-24- *Effects*: Equivalent to `insert(p - begin(), basic_string(first, last, _get_allocator()))`.

-25- *Returns*: An iterator which refers to the copy of the first inserted character, or `p` if `first == last`.

4. In 21.3.2.6.6 [string.replace], add the allocator parameter to the range overload temporary:

```
template<class InputIterator>
basic_string& replace(const_iterator i1, const_iterator i2,
                    InputIterator j1, InputIterator j2);
```

-32- *Requires*: `[begin(), i1)`, `[i1, i2)` and `[j1, j2)` are valid ranges.

-33- *Effects*: Calls `replace(i1 - begin(), i2 - i1, basic_string(j1, j2, _get_allocator()))`.

-34- *Returns*: `*this`.

2801. Default-constructibility of `unique_ptr`

Section: 20.11.1.2.1 [unique.ptr.single.ctor] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2016-11-09 **Last modified:** 2017-03-03

Priority: 2

View other [active issues](#) in [unique.ptr.single.ctor].

View all other [issues](#) in [unique.ptr.single.ctor].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 122

`unique_ptr` should not satisfy `is_constructible_v<unique_ptr<T, D>>` unless `D` is `DefaultConstructible` and not a pointer type. This is important for interactions with `pair`, `tuple`, and `variant` constructors that rely on the `is_default_constructible` trait.

Suggested resolution:

Add a *Remarks*: clause to constrain the default constructor to not exist unless the *Requires* clause is satisfied.

[Issues Telecon 16-Dec-2016]

Priority 2; Howard and Ville to provide wording.

[2016-12-24: Howard and Ville provided wording.]

[2017-03-02, Kona, STL comments and tweaks the wording]

LWG discussed this issue, and we liked it, but we wanted to tweak the PR. I ran this past Ville (who drafted the PR with Howard), and he was in favor of tweaking this.

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

1. Modify the synopsis in 20.11.1.2 [unique.ptr.single] as follows:

```
[...]
constexpr unique_ptr(nullptr_t) noexcept;
: unique_ptr() { }
[...]
```

2. Modify 20.11.1.2.1 [unique.ptr.single.ctor] as follows:

```
constexpr unique_ptr() noexcept;
constexpr unique_ptr(nullptr_t) noexcept;
```

-1- *Requires*: `D` shall satisfy the requirements of `DefaultConstructible` (Table 22), and that construction shall not throw an exception.

-2- *Effects*: Constructs a `unique_ptr` object that owns nothing, value-initializing the stored pointer and the stored deleter.

-3- *Postconditions*: `get() == nullptr.get_deleter()` returns a reference to the stored deleter.

-4- *Remarks*: If this constructor is instantiated with a pointer type or reference type for the template argument `D`, the program is ill-formed. If `is_pointer_v<deleter_type>` is true OR `is_default_constructible_v<deleter_type>` is false, this constructor shall not participate in overload resolution.

```
explicit unique_ptr(pointer p) noexcept;
```

[...]

-8- Remarks: If this constructor is instantiated with a pointer type or reference type for the template argument D, the program is ill-formed. If is_pointer v<deleter type> is true OR is default constructible v<deleter type> is false, this constructor shall not participate in overload resolution.

3. Modify the synopsis in 20.11.1.3 [unique.ptr.runtime] as follows:

```
[...]
constexpr unique_ptr(nullptr_t) noexcept; : unique_ptr() { }
[...]
```

4. Modify 20.11.1.3.1 [unique.ptr.runtime.ctor] as follows:

```
template <class U> explicit unique_ptr(U p) noexcept;
```

This constructor behaves the same as the constructor that takes a pointer parameter in the primary template except that the following constraints are added for it to participate in overload resolution

- U is the same type as pointer, or
- pointer is the same type as element_type*, U is a pointer type V*, and V(*)[] is convertible to element_type(*)[].

```
template <class U> unique_ptr(U p, see below d) noexcept;
template <class U> unique_ptr(U p, see below d) noexcept;
```

1 These constructors behave the same as the constructors that take a pointer parameter in the primary template except that they shall not participate in overload resolution unless either

2802. shared_ptr constructor requirements for a deleter

Section: 20.11.2.2.1 [util.smartptr.shared.const] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2016-11-09 **Last modified:** 2017-03-03

Priority: 2

View other [active issues](#) in [util.smartptr.shared.const].

View all other [issues](#) in [util.smartptr.shared.const].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 127

It should suffice for the deleter D to be nothrow move-constructible. However, to avoid potentially leaking the pointer p if D is also copy-constructible when copying the argument by-value, we should continue to require the copy constructor does not throw if D is CopyConstructible.

Proposed change:

Relax the requirement the D be CopyConstructible to simply require that D be MoveConstructible. Clarify the requirement that construction of any of the arguments passed by-value shall not throw exceptions. Note that we have library-wide wording in clause 17 that says any type supported by the library, not just this delete, shall not throw exceptions from its destructor, so that wording could be editorially removed. Similarly, the requirements that A shall be an allocator satisfy that neither constructor nor destructor for A can throw.

[2016-12-16, Issues Telecon]

Priority 3; Jonathan to provide wording.

[2017-02-23, Jonathan comments and suggests wording]

I don't think the Clause 17 wording in [res.on.functions] is sufficient to require that the delete expression is well-formed. A class-specific deallocation function ([class.free]) would not be covered by [res.on.functions] and so could throw:

```
struct Y { void operator delete(void*) noexcept(false) { throw 1; } };
```

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
```

```
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- Requires: `D` shall be `CopyMoveConstructible` and `such` construction of `d` and a deleter of type `D` initialized with `std::move(d)` shall not throw exceptions. The destructor of `D` shall not throw exceptions. The expression `d(p)` shall be well formed, shall have well-defined behavior, and shall not throw exceptions. `A` shall be an allocator (17.5.3.5). The copy constructor and destructor of `A` shall not throw exceptions. When `τ` is [...].

2806. Base class of `bad_optional_access`

Section: 20.6.5 [optional.bad.access] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2016-11-09 **Last modified:** 2017-03-03

Priority: 1

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 49

[LEWG 72](#) suggests changing the base class of `std::bad_optional_access`, but the issue appears to have been forgotten.

Proposed change:

Address LEWG issue 72, either changing it for C++17 or closing the issue.

[Issues Telecon 16-Dec-2016]

Priority 1; Marshall provides wording

Not the most important bug in the world, but we can't change it after we ship C++17, hence the P1.

[Kona 2017-03-01]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

Changes are relative to N4604.

In the class synopsis in 20.6.5 [optional.bad_optional_access] change:

```
class bad_optional_access : public exceptionlogic_error {
```

2807. `std::invoke` should use `std::is_nothrow_callable`

Section: 20.14.4 [func.invoke] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2016-11-09 **Last modified:** 2017-03-03

Priority: 3

View other [active issues](#) in [func.invoke].

View all other [issues](#) in [func.invoke].

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 53

`std::invoke` can be made trivially noexcept using the new `std::is_nothrow_callable` trait:

Proposed change:

Add the exception specifier `noexcept(is_nothrow_callable_v<F&&(Args&&...)>)` to `std::invoke`.

[Issues Telecon 16-Dec-2016]

Priority 3

[2017-02-28, Daniel comments and provides wording]

The following wording *assumes* that D0604R0 would be accepted, therefore uses `is_nothrow_invocable_v` instead of the suggested current trait `is_nothrow_callable_v`

[Kona 2017-03-01]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Change 20.14.1 [functional.syn], header <functional> synopsis, as indicated:

```
// 20.14.4, invoke
template <class F, class... Args>
    invoke_result_t<F, Args...> result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args) noexcept(is_nothrow_invocable_v<F, Args...>);
```

2. Change 20.14.4 [func.invoke] as indicated:

```
template <class F, class... Args>
    invoke_result_t<F, Args...> result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args) noexcept(is_nothrow_invocable_v<F, Args...>);
```

[2857](#). {variant,optional,any}::emplace should return the constructed value

Section: 20.6 [optional], 20.7 [variant], 20.8 [any] **Status:** [Immediate](#) **Submitter:** Zhihao Yuan **Opened:** 2017-01-25 **Last modified:** 2017-03-03

Priority: 1

View all other [issues](#) in [optional].

View all issues with [Immediate](#) status.

Discussion:

When you want to continue operate on the new contained object constructed by `emplace`, you probably don't want to go through the type-safe machinery again.

[2017-01-27 Telecon]

Priority 1; send to LEWG.

[Kona 2017-03-02]

Accepted as Immediate because P1.

Proposed resolution:

This wording is relative to [N4618](#).

1. Update the following signatures in 20.6.3 [optional.optional], class template `optional` synopsis, as indicated:

```
[...]
// 20.6.3.3 [optional.assign], assignment
[...]
```

```
template <class... Args> voidT& emplace(Args&&...);
template <class U, class... Args>
    voidT& emplace(initializer_list<U>, Args&&...);
[...]
```

2. Modify 20.6.3.3 [optional.assign] as indicated:

```
template <class... Args> voidT& emplace(Args&&... args);

[...]
```

-27- *Postconditions*: `*this` contains a value.

-?- Returns: A reference to the new contained value.

-28- *Throws*: Any exception thrown by the selected constructor of `T`.

```
template <class U, class... Args> voidT& emplace(initializer_list<U> il, Args&&... args);

[...]
```

-31- *Postconditions*: `*this` contains a value.

-?- Returns: A reference to the new contained value.

-32- *Throws*: Any exception thrown by the selected constructor of `T`.

3. Modify the following signatures in 20.7.3 [variant.variant], class template `variant` synopsis, as indicated:

```
[...]
// 20.7.3.4 [variant.mod], modifiers
template <class T, class... Args> voidT& emplace(Args&&...);
template <class T, class U, class... Args>
    voidT& emplace(initializer_list<U>, Args&&...);
template <size_t I, class... Args> void variant_alternative_t<I, variant<Types...>& emplace(Args&&...);
```

```
template <size_t I, class U, class... Args>
void variant_alternative_t<I, variant<Types...>>& emplace(initializer_list<U>, Args&&...);
[...]
```

4. Modify 20.7.3.4 [variant.mod] as indicated:

```
template <class T, class... Args> void T& emplace(Args&&... args);
```

-1- *Effects*: Equivalent to `return emplace<I>(std::forward<Args>(args)...)`; where I is the zero-based index of τ in `Types...`.

[...]

```
template <class T, class U, class... Args>
void T& emplace(initializer_list<U> il, Args&&... args);
```

-3- *Effects*: Equivalent to `return emplace<I>(il, std::forward<Args>(args)...)`; where I is the zero-based index of τ in `Types...`.

[...]

```
template <size_t I, class... Args> void variant_alternative_t<I, variant<Types...>>& emplace(Args&&... args);
```

[...]

-7- *Postconditions*: `index()` is I .

-?- *Returns*: A reference to the new contained value.

-8- *Throws*: Any exception thrown during the initialization of the contained value.

[...]

```
template <size_t I, class U, class... Args>
void variant_alternative_t<I, variant<Types...>>& emplace(initializer_list<U> il, Args&&... args);
```

[...]

-12- *Postconditions*: `index()` is I .

-?- *Returns*: A reference to the new contained value.

-13- *Throws*: Any exception thrown during the initialization of the contained value.

[...]

5. Modify the following signatures in 20.8.3 [any.class], class any synopsis, as indicated:

```
[...]
// 20.8.3.3 [any.modifiers], modifiers
template <class ValueType, class... Args>
void decay_t<ValueType>& emplace(Args&& ...);
template <class ValueType, class U, class... Args>
void decay_t<ValueType>& emplace(initializer_list<U>, Args&&...);
[...]
```

6. Modify 20.8.3.3 [any.modifiers] as indicated:

```
template <class ValueType, class... Args>
void decay_t<ValueType>& emplace(Args&&... args);
```

[...]

-4- *Postconditions*: `*this` contains a value.

-?- *Returns*: A reference to the new contained value.

-5- *Throws*: Any exception thrown by the selected constructor of τ .

[...]

```
template <class ValueType, class U, class... Args>
void decay_t<ValueType>& emplace(initializer_list<U> il, Args&&... args);
```

[...]

-10- *Postconditions*: `*this` contains a value.

-?- *Returns*: A reference to the new contained value.

-11- *Throws*: Any exception thrown by the selected constructor of τ .

[...]

2861. basic_string should require that charT match traits::char_type

Section: 21.3.2.1 [string.require] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-02 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [string.require].

View all other [issues](#) in [string.require].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 145

There is no requirement that `traits::char_type` is `charT`, although there is a requirement that `allocator::value_type` is `charT`. This means that it might be difficult to honour both methods returning reference (such as `operator[]`) and `charT&` (like `front/back`) when `traits` has a surprising `char_type`. It seems that the allocator should *not* rebound in such cases, making the reference-returning signatures the problematic ones.

Suggested resolution: Add a requirement that `is_same_v<typename traits::char_type, charT>` is true, and simplify so that `value_type` is just an alias for `charT`.

[2017-02-02 Marshall adds]

In [string.require]/3, there's already a note that the types shall be the same. In [string.view.template]/1, it says "In every specialization `basic_string_view<charT, traits, Allocator>`, the type `traits` shall satisfy the character traits requirements (21.2), and the type `traits::char_type` shall name the same type as `charT`".

[Kona 2017-02-28]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

Changes are based off of [N4618](#)

1. Modify [basic.string] as indicated (in the synopsis):

```
class basic_string {
public:
    // types:
    using traits_type      = traits;
    using value_type       = charTtypename traits::char_type;
    using allocator_type    = Allocator
```

2. Change [string.require]/3 as indicated:

-3- In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT`. Every object of type `basic_string<charT, traits, Allocator>` shall use an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used shall be obtained as described in 23.2.1. **[Note:** In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall satisfy the character traits requirements (21.2), and the type `traits::char_type` shall name the same type as `charT`; ~~see 21.2. — end note~~ **]**

2866. Incorrect derived classes constraints

Section: 17.5.5.11 [derivation] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 36

For bullet (3.2), no base classes are described as non-virtual. Rather, base classes are not specified as virtual, a subtly different negative.

Proposed change:

Rewrite bullet 3.2:

Every base class not specified as virtual shall not be virtual;

[Kona 2017-03-01]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 17.5.5.11 [derivation] paragraph 3.2 as indicated:

~~Every base class described as non-virtual shall not be virtual;~~ Every base class not specified as virtual shall not be virtual;

[2868](#). Missing specification of `bad_any_cast::what()`

Section: 20.8.2 [any.bad_any_cast] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 54

There is no specification for `bad_any_cast.what`.

Proposed change:

Add a paragraphs:

```
const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.

Remarks: The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4).

[Kona 2017-03-01]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4618](#).

1. Insert the following series of paragraphs to 20.8.2 [any.bad_any_cast] as indicated:

`const char* what() const noexcept override;`

[-? Returns: An implementation-defined NTBS.]

[-? Remarks: The message may be a null-terminated multibyte string (17.4.2.1.5.2 [multibyte.strings]), suitable for conversion and display as a `wstring` (21.3 [string.classes], 22.4.1.4 [locale.codecvt]).]

[2872](#). Add definition for direct-non-list-initialization

Section: 17.3 [definitions] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all issues with [Immediate](#) status.

Discussion:

Addresses US 107

The term 'direct non-list initialization' needs to be incorporated from the Library Fundamentals TS, as several components added to C++17 rely on this definition.

Proposed change:

Add:

17.3.X direct-non-list-initialization [defns.direct-non-list-init]

A direct-initialization that is not list-initialization.

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4618](#).

1. Add the following to 17.3 [definitions] as indicated:

17.3.? direct-non-list-initialization [defs.direct-non-list-init]

A direct-initialization that is not list-initialization.

2873. Add noexcept to several shared_ptr related functions

Section: 20.11.2.2 [util.smartptr.shared] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [util.smartptr.shared].

View all other [issues](#) in [util.smartptr.shared].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 124

Several shared_ptr related functions have wide contracts and cannot throw, so should be marked unconditionally noexcept.

Proposed change:

Add noexcept to:

```
template<class U>
bool
shared_ptr::owner_before(shared_ptr<U> const& b) const noexcept;

template<class U>
bool
shared_ptr::owner_before(weak_ptr<U> const& b) const noexcept;

template<class U>
bool
weak_ptr::owner_before(shared_ptr<U> const& b) const noexcept;

template<class U>
bool
weak_ptr::owner_before(weak_ptr<U> const& b) const noexcept;

bool owner_less::operator()(A,B) const noexcept; // all versions
```

[2017-02-20, Marshall adds wording]

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.11.2.2 [util.smartptr.shared] as indicated:

```
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

2. Modify 20.11.2.2.5 [util.smartptr.shared.obs] as indicated:

```
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

3. Modify 20.11.2.4 [util.smartptr.ownerless] as indicated:

```
template<class T> struct owner_less<shared_ptr<T>> {
    bool operator()(const shared_ptr<T>&, const shared_ptr<T>&) const noexcept;
    bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
    bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
};

template<class T> struct owner_less<weak_ptr<T>> {
    bool operator()(const weak_ptr<T>&, const weak_ptr<T>&) const noexcept;
    bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
    bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
};

template<> struct owner_less<void> {
    template<class T, class U>
        bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;
```

```

template<class T, class U>
    bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;
template<class T, class U>
    bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;
template<class T, class U>
    bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;
using is_transparent = unspecified;
};

```

2874. Constructor `shared_ptr::shared_ptr(Y*)` should be constrained

Section: 20.11.2.2.1 [util.smartptr.shared.const] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [util.smartptr.shared.const].

View all other [issues](#) in [util.smartptr.shared.const].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 125

Paragraph 4: This constructor should not participate in overload resolution unless the *Requires* clause is satisfied. Note that this would therefore apply to some assignment operator and `reset` overloads, via *Effects*: equivalent to some code wording.

Proposed change:

Add a *Remarks*: clause to constrain this constructor not to participate in overload resolution unless the *Requires* clause is satisfied.

[2017-02-23, Jonathan provides wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

[Drafting note: This also adds a hyphen to "well defined"]

```

template<class Y> explicit shared_ptr(Y* p);

```

-4- *Requires*: Υ shall be a complete type. The expression `delete[] p`, when Υ is an array type, or `delete p`, when Υ is not an array type, shall be well-formed, shall have well-defined behavior, and shall not throw exceptions. When Υ is $\mathcal{U}[N]$, $\Upsilon(*)[N]$ shall be convertible to Υ^* ; when Υ is $\mathcal{U}[]$, $\Upsilon(*)[]$ shall be convertible to Υ^* ; otherwise, Υ^* shall be convertible to Υ^* .

-5- *Effects*: [...]

-6- *Postconditions*: [...]

-7- *Throws*: [...]

-?- *Remarks*: When Υ is an array type, this constructor shall not participate in overload resolution unless the expression `delete[] p` is well-formed and either Υ is $\mathcal{U}[N]$ and $\Upsilon(*)[N]$ is convertible to Υ^* , or $\Upsilon(*)[]$ is convertible to Υ^* . When Υ is not an array type, this constructor shall not participate in overload resolution unless the expression `delete p` is well-formed and Υ^* is convertible to Υ^* .

[Kona 2017-02-27: Jonathan updates wording after LWG review]

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

[Drafting note: This also adds a hyphen to "well defined"]

```

template<class Y> explicit shared_ptr(Y* p);

```

-4- *Requires*: $\forall$ shall be a complete type. The expression `delete[] p`, when τ is an array type, or `delete p`, when τ is not an array type, shall be well-formed, shall have well-defined behavior, and shall not throw exceptions. When τ is $U[N]$, $\forall(*)[N]$ shall be convertible to τ^* ; when τ is $U[]$, $\forall(*)[]$ shall be convertible to τ^* ; otherwise, $\forall^*$ shall be convertible to τ^* .

-5- *Effects*: [...]

-6- *Postconditions*: [...]

-7- *Throws*: [...]

-?- *Remarks*: When τ is an array type, this constructor shall not participate in overload resolution unless the expression `delete[] p` is well-formed and either τ is $U[N]$ and $\forall(*)[N]$ is convertible to τ^* , or τ is $U[]$ and $\forall(*)[]$ is convertible to τ^* . When τ is not an array type, this constructor shall not participate in overload resolution unless the expression `delete p` is well-formed and $\forall^*$ is convertible to τ^* .

2875. `shared_ptr::shared_ptr(Y*, D, [...])` constructors should be constrained

Section: 20.11.2.2.1 [util.smartptr.shared.const] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [util.smartptr.shared.const].

View all other [issues](#) in [util.smartptr.shared.const].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 126

Paragraph 8: This constructor should not participate in overload resolution unless the *Requires* clause is satisfied. Note that this would therefore apply to some assignment operator and `reset` overloads, via *Effects*: equivalent to some code wording.

Proposed change:

Add a *Remarks*: clause to constrain this constructor not to participate in overload resolution unless the *Requires* clause is satisfied.

[2017-02 pre-Kona]

See US 125: LWG [2874](#).

[2017-02-23, Jonathan provides wording]

This wording is relative to [N4640](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

A. If the proposed resolution of LWG [2802](#) has been accepted modify p8 as shown:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- *Requires*: D shall be MoveConstructible and e Construction of d and a deleter of type D initialized with `std::move(d)` shall not throw exceptions. The expression `d(p)` shall be well-formed, shall have well-defined behavior, and shall not throw exceptions. A shall be an allocator (17.5.3.5). When τ is $U[N]$, $\forall(*)[N]$ shall be convertible to τ^* ; when τ is $U[]$, $\forall(*)[]$ shall be convertible to τ^* ; otherwise, $\forall^*$ shall be convertible to τ^* .

B. If the proposed resolution of LWG [2802](#) has not been accepted modify p8 as shown:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- *Requires*: D shall be CopyConstructible and such e Construction of d and a copy of d shall not throw exceptions. The destructor of D shall not throw exceptions. The expression `d(p)` shall be well-formed, shall have well-defined behavior, and shall not throw exceptions. A shall be an allocator (17.5.3.5). The copy constructor and destructor of A shall not throw exceptions. When τ is $U[N]$, $\forall(*)[N]$ shall be convertible to τ^* ; when τ is $U[]$, $\forall(*)[]$ shall be convertible to τ^* ; otherwise, $\forall^*$ shall be convertible to τ^* .

1. In either case, add a *Remarks* paragraph after p11:

[Drafting note: If LWG [2802](#) is not accepted, replace `is_move_constructible_v` with `is_copy_constructible_v`.]

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- *Requires*: [...]

[...]

-11- *Throws*: [...]

-?- *Remarks*: When τ is an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and either τ is $U[N]$ and $Y(*)[N]$ is convertible to τ^* , or $Y(*)[]$ is convertible to τ^* . When τ is not an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and Y^* is convertible to τ^* .

[Kona 2017-02-27: Jonathan updates wording after LWG review]

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#) as modified by the proposed resolution of LWG [2802](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- *Requires*: ~~D shall be MoveConstructible and c~~Construction of `d` and a deleter of type `D` initialized with `std::move(d)` shall not throw exceptions. The expression `d(p)` ~~shall be well-formed~~, shall have well-defined behavior, and shall not throw exceptions. `A` shall be an allocator (17.5.3.5). ~~When τ is $U[N]$, $Y(*)[N]$ shall be convertible to τ^* ; when τ is $U[]$, $Y(*)[]$ shall be convertible to τ^* ; otherwise, Y^* shall be convertible to τ^* .~~

2. Add a *Remarks* paragraph after p11:

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- *Requires*: [...]

[...]

-11- *Throws*: [...]

-?- *Remarks*: When τ is an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and either τ is $U[N]$ and $Y(*)[N]$ is convertible to τ^* , or τ is $U[]$ and $Y(*)[]$ is convertible to τ^* . When τ is not an array type, this constructor shall not participate in overload resolution unless `is_move_constructible_v<D>` is true, the expression `d(p)` is well-formed, and Y^* is convertible to τ^* .

[2876](#). `shared_ptr::shared_ptr(const weak_ptr<Y>&)` constructor should be constrained

Section: 20.11.2.2.1 [util.smartptr.shared.const] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [util.smartptr.shared.const].

View all other [issues](#) in [util.smartptr.shared.const].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 129

Paragraph 22: This constructor should not participate in overload resolution unless the requirements are satisfied, in order to give correct results from the `is_constructible` trait.

Proposed change:

Add a *Remarks*: clause to constrain this constructor not to participate in overload resolution unless the *Requires* clause is satisfied.

[2017-02-23, Jonathan provides wording]

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.11.2.2.1 [util.smartptr.shared.const] as indicated:

```
template<class Y> explicit shared_ptr(const weak_ptr<Y>& r);
```

~~-22- Requires: Y^* shall be compatible with τ^* .~~

[...]

-25- Throws: [...]

-?- Remarks: This constructor shall not participate in overload resolution unless Y^* is compatible with τ^* .

[2878](#). Missing DefaultConstructible requirement for istream_iterator default constructor

Section: 24.6.1.1 [istream.iterator.cons] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [istream.iterator.cons].

View all other [issues](#) in [istream.iterator.cons].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 153

istream_iterator default constructor requires a DefaultConstructible T.

Proposed change:

Add a new p1:

Requires: T is DefaultConstructible.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4618](#).

1. Modify 24.6.1.1 [istream.iterator.cons] as indicated:

```
see below istream_iterator();
```

-?- Requires: T is DefaultConstructible.

-1- *Effects:* Constructs the end-of-stream iterator. If `is_trivially_default_constructible_v<T>` is true, then this constructor is a constexpr constructor.

-2- *Postconditions:* `in_stream == 0`.

[Kona 2017-02-28: Jonathan provides updated wording as requested by LWG.]

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4618](#).

1. Modify 24.6.1 [istream.iterator] as indicated:

-1- The class template `istream_iterator` is an input iterator (24.2.3) that reads (using `operator>>`) successive elements from the input stream for which it was constructed. After it is constructed, and every time `++` is used, the iterator reads and stores a value of τ . If the iterator fails to read and store a value of τ (`fail()` on the stream returns true), the iterator becomes equal to the end-of-stream iterator value. The constructor with no arguments `istream_iterator()` always constructs an end-of-stream input iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-stream iterator is not defined. For any other iterator value a const $\tau\&$ is returned. The result of `operator->` on an end-of-stream iterator is

not defined. For any other iterator value a `const T*` is returned. The behavior of a program that applies `operator++()` to an end-of-stream iterator is undefined. It is impossible to store things into istream iterators. The type `T` shall meet the `DefaultConstructible`, `CopyConstructible`, and `CopyAssignable` requirements.

2890. The definition of 'object state' applies only to class types

Section: 17.3.16 [defns.obj.state] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all other [issues](#) in [defns.obj.state].

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 30

The definition of 'object state' applies only to class types, implying that fundamental types and arrays do not have this property.

Proposed change: Replacing "an object state" with "a value of an object" in 17.3.25 [defns.valid] and dropping the definition of "object state" in 17.3.16 [defns.obj.state].

[2017-02-26, Scott Schurr provides wording]

[Kona 2017-03-01]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 17.3.16 [defns.obj.state] as indicated:

17.3.16 [defns.obj.state]

object state

~~the current value of all non-static class members of an object (9.2) [Note: The state of an object can be obtained by using one or more observer functions. — end note]~~

2. Modify 17.3.25 [defns.valid] as indicated:

17.3.25 [defns.valid]

valid but unspecified state

~~an object state~~ **a value of an object** that is not specified except that the object's invariants are met and operations on the object behave as specified for its type

2894. The function template `std::apply()` is required to be `constexpr`, but `std::invoke()` isn't

Section: 20.14.4 [func.invoke] **Status:** [Immediate](#) **Submitter:** Great Britain **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [func.invoke].

View all other [issues](#) in [func.invoke].

View all issues with [Immediate](#) status.

Discussion:

Addresses GB 51

The function template `std::apply()` in 20.5.3.5 [tuple.apply] is required to be `constexpr`, but `std::invoke()` in 20.14.4 [func.invoke] isn't. The most sensible implementation of `apply_impl()` is exactly equivalent to `std::invoke()`, so this requires implementations to have a `constexpr` version of `invoke()` for internal use, and the public API `std::invoke`, which must not be `constexpr` even though it is probably implemented in terms of the internal version.

Proposed change: Add `constexpr` to `std::invoke`.

[2017-02-20, Marshall adds wording]

[Kona 2017-03-01]

We think this needs CWG 1581 to work; accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.14.4 [func.invoke] as indicated:

```
template <class F, class... Args>
    constexpr result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args);
```

[2900](#). The copy and move constructors of optional are not constexpr

Section: 20.6.3 [optional.optional] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [optional.optional].

View all other [issues](#) in [optional.optional].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 111

The copy and move constructors of optional are not constexpr. However, the constructors taking a const T& or T&& are constexpr, and there is a precedent for having a constexpr copy constructor in 26.5.2 [complex]. The defaulted copy and move constructors of pair and tuple are also conditionally constexpr (see 20.4.2 [pairs.pair] p2 and 20.5.2.1 [tuple.cnstr] p2).

A strong motivating use-case is constexpr functions returning optional values. This issue was discovered while working on a library making heavy use of such.

Proposed change: Add constexpr to:

```
optional(const optional &);
optional(optional &&) noexcept(see below);
```

[2017-02-23, Casey comments and suggests wording]

This issue corresponds to NB comment US 111, which requests that the move and copy constructors of std::optional be declared constexpr. The PR simply suggests adding the constexpr specifier to the declarations of the constructors. The PR fails to specify the most important thing — and this has been a failing of Library in general — under what conditions is the thing that we've declared constexpr actually expected to be usable in constant expression context? (I think the proper standardese here is "under what conditions is the full expression of an initialization that would invoke these constructors a constant subexpression?")

It is, I believe, well-known that conforming implementations of optional<T> must store a T in a union to provide constexpr constructors that either do [optional(T const&)] or do not [optional()] initialize the contained T. A general implementation of optional's copy/move constructors must statically choose which union member, if any, to activate in each constructor. Since there is no way to change the active member of a union in a constant expression, and a constructor must *statically* choose a union member to activate (i.e., without being affected by the runtime state of the copy/move constructor's argument) it's not possible to implement a *general* constexpr copy/move constructor for optional.

It is, however, possible to copy/move construct a trivially copy constructible/trivially move constructible union in constexpr context, which effectively copies the union's object representation, resulting in a union whose active member is the same as the source union's. Indeed, at least two major implementations of optional (MSVC and libc++) already provide that behavior as a conforming optimization when T is a trivially copyable type. If we are to declare optional<T>'s copy and move constructors constexpr, we should additionally specify that those constructors are only required to have the "constexpr mojo" when T is trivially copyable. (Note that I suggest "trivially copyable" here rather than "trivially copy constructible or trivially move constructible" since the simpler requirement is simpler to implement, and I don't believe the more complicated requirement provides any additional benefit: I've never seen a trivially copy constructible or trivially move constructible type outside of a test suite that was not also trivially copyable.)

Previous resolution [SUPERSEDED]:

This wording is relative to [N4618](#).

1. Edit 20.6.3 [optional.optional] as indicated:

```
constexpr optional(const optional &);
constexpr optional(optional &&) noexcept(see below);
```

2. Edit 20.6.3.1 [optional.ctor] paragraph as indicated:

```
constexpr optional(const optional &);
```

and

```
constexpr optional(optional &&) noexcept(see below);
```

[2017-02-23, Marshall comments]

This is related to LWG [2745](#).

[2017-02-28, Kona, Casey comments and improves wording]

Amended PR per LWG discussion in Kona: replace the "is trivially copyable" requirement with the more specific `is_trivially_copy/move_constructible<T>`. LWG was concerned that tuple is a counter-example to the assumption that all three traits are equivalent for real-world types.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Change the synopsis of class template `optional` in 20.6.3 [optional.optional] as follows:

```
[...]  
// 20.6.3.1, constructors  
constexpr optional() noexcept;  
constexpr optional(nullopt_t) noexcept;  
constexpr optional(const optional&);  
constexpr optional(optional&&) noexcept(see below);  
[...]
```

2. Modify 20.6.3.1 [optional.ctor] as indicated:

```
constexpr optional(const optional& rhs);
```

[...]

-6- *Remarks:* This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is true. **If `T` is a trivially copyable type, this constructor shall be a constexpr constructor.**

```
constexpr optional(optional&& rhs) noexcept(see below);
```

[...]

-10- *Remarks:* The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is true. **If `T` is a trivially copyable type, this constructor shall be a constexpr constructor.**

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Change the synopsis of class template `optional` in 20.6.3 [optional.optional] as follows:

```
[...]  
// 20.6.3.1, constructors  
constexpr optional() noexcept;  
constexpr optional(nullopt_t) noexcept;  
constexpr optional(const optional&);  
constexpr optional(optional&&) noexcept(see below);  
[...]
```

2. Modify 20.6.3.1 [optional.ctor] as indicated:

```
constexpr optional(const optional& rhs);
```

[...]

-6- *Remarks:* This constructor shall be defined as deleted unless `is_copy_constructible_v<T>` is true. **If `T` is trivially copyable, this constructor shall be a constexpr constructor.**

```
constexpr optional(optional&& rhs) noexcept(see below);
```

[...]

-10- *Remarks:* The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible_v<T>`. This constructor shall not participate in overload resolution unless `is_move_constructible_v<T>` is true. **If `T` is trivially moveable, this constructor shall be a constexpr constructor.**

Priority: Not Prioritized

View other [active issues](#) in [variant.ctor].

View all other [issues](#) in [variant.ctor].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 118

The form of initialization for the emplace-constructors is not specified. We are very clear to mandate "as if by direct non-list initialization" for each constructor in optional, so there is no ambiguity regarding parens vs. braces. That wording idiom should be followed by variant.

Proposed change: Insert the phrase "as if direct-non-list-initializing" at appropriate locations in paragraphs 19, 23, 27, and 31

[2017-02-20, Marshall adds wording]

[2017-02-27, Marshall adds wording to cover two more cases]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Modify 20.7.3.1 [variant.ctor] paragraph 19 as indicated:

Effects: Initializes the contained value [as if direct-non-list-initializing an object](#) of type τ with the arguments `std::forward<Args>(args)...`

2. Modify 20.7.3.1 [variant.ctor] paragraph 23 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ with the arguments `il, std::forward<Args>(args)...`

3. Modify 20.7.3.1 [variant.ctor] paragraph 27 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ_1 with the arguments `std::forward<Args>(args)...`

4. Modify 20.7.3.1 [variant.ctor] paragraph 31 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ_1 with the arguments `il, std::forward(args)...`

[Kona 2017-02-28]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.7.3.1 [variant.ctor] paragraph 19 as indicated:

Effects: Initializes the contained value [as if direct-non-list-initializing an object](#) of type τ with the arguments `std::forward<Args>(args)...`

2. Modify 20.7.3.1 [variant.ctor] paragraph 23 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ with the arguments `il, std::forward<Args>(args)...`

3. Modify 20.7.3.1 [variant.ctor] paragraph 27 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ_1 with the arguments `std::forward<Args>(args)...`

4. Modify 20.7.3.1 [variant.ctor] paragraph 31 as indicated:

Effects: Initializes the contained value as if [direct-non-list-initializingconstructing](#) an object of type τ_1 with the arguments `il, std::forward(args)...`

5. Modify 20.7.3.4 [variant.mod] paragraph 6 as indicated:

Effects: Destroys the currently contained value if `valueless_by_exception()` is false. Then [direct-](#)initializes the contained value as

if ~~direct-non-list-initializing~~ constructing a value of type T_i with the arguments `std::forward<Args>(args)...`

6. Modify 20.7.3.4 [variant.mod] paragraph 11 as indicated:

Effects: Destroys the currently contained value if `valueless_by_exception()` is false. Then ~~direct-~~ initializes the contained value as if ~~direct-non-list-initializing~~ constructing a value of type T_i with the arguments `il, std::forward<Args>(args)...`

2904. Make variant move-assignment more exception safe

Section: 20.7.3.3 [variant.assign] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all issues with [Immediate](#) status.

Discussion:

Addresses US 119 and CH 7

The copy-assignment operator is very careful to not destroy the contained element until after a temporary has been constructed, which can be safely moved from.

This makes the `valueless_by_exception` state extremely rare, by design.

However, the same care and attention is not paid to the move-assignment operator, nor the assignment-from-deduced-value assignment template. This concern should be similarly important in these cases, especially the latter.

Proposed change: —

[2017-03-02, Kona, Casey comments and suggests wording]

The wording below has been developed with *much* input from Tomasz.

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.7.3.3 [variant.assign] as indicated:

[Drafting note: Presentation of para 9 immediately below has been split into individual bullets.]

```
variant& operator=(const variant& rhs);
```

Let j be `rhs.index()`.

-1- *Effects:*

(1.1) — If neither `*this` nor `rhs` holds a value, there is no effect. Otherwise,

(1.2) — if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value. Otherwise,

(1.3) — if `index() == jrhs.index()`, assigns the value contained in `rhs` to the value contained in `*this`. Otherwise,

(1.?) — if `is_nothrow_copy_constructible_v<Tj> || !is_nothrow_move_constructible_v<Tj>` is true, equivalent to `emplace<j>(get<j>(rhs))`. Otherwise,

(1.4) — equivalent to `operator=(variant(rhs))` copies the value contained in `rhs` to a temporary, then destroys any value contained in `*this`. Sets `*this` to hold the same alternative index as `rhs` and initializes the value contained in `*this` as if `direct-non-list-initializing` an object of type T_j with `std::forward<Tj>(TMP)`, with `TMP` being the temporary and j being `rhs.index()`.

-2- *Returns:* `*this`.

-3- *Postconditions:* `index() == rhs.index()`.

-4- *Remarks:* This function shall not participate in overload resolution unless `is_copy_constructible_v<Tj> && is_move_constructible_v<Tj> && is_copy_assignable_v<Tj>` is true for all i .

(4.1) — If an exception is thrown during the call [...]

(4.2) — If an exception is thrown during the call [...]

(4.3) — If an exception is thrown during the call [...]

```
variant& operator=(variant&& rhs) noexcept(see below);
```

Let j be `rhs.index()`.

-5- Effects:

(5.1) — If neither `*this` nor `rhs` holds a value, there is no effect. Otherwise,

(5.2) — if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value. Otherwise,

(5.3) — if `index() == rhs.index()`, assigns `get<j>(std::move(rhs))` to the value contained in `*this`, with j being `index()`. Otherwise,

(5.4) — `emplace<j>(get<j>(std::move(rhs)))` destroys any value contained in `*this`. Sets `*this` to hold the same alternative index as `rhs` and initializes the value contained in `*this` as if direct-non-list-initializing an object of type T_j with `get<j>(std::move(rhs))` with j being `rhs.index()`.

[...]

[...]

```
template <class T> variant& operator=(T&& t) noexcept(see below);
```

-8- [...]

-9- Effects:

(9.1) — If `*this` holds a T_j , assigns `std::forward<T>(t)` to the value contained in `*this`. Otherwise,

(9.2) — if `is_nothrow_constructible_v<Tj, T> || !is_nothrow_move_constructible_v<Tj>` is true, equivalent to `emplace<j>(std::forward<T>(t))`. Otherwise,

(9.3) — `operator=(variant(std::forward<T>(t)))` destroys any value contained in `*this`, sets `*this` to hold the alternative type T_j as selected by the imaginary function overload resolution described above, and direct-initializes the contained value as if direct-non-list-initializing it with `std::forward<T>(t)`.

2905. `is_constructible_v<unique_ptr<P, D>, P, D const &>` should be false when `D` is not copy constructible

Section: 20.11.1.2.1 [unique.ptr.single.ctor] Status: [Immediate](#) Submitter: United States Opened: 2017-02-03 Last modified: 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [unique.ptr.single.ctor].

View all other [issues](#) in [unique.ptr.single.ctor].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 123

`is_constructible_v<unique_ptr<P, D>, P, D const &>` should be false when `D` is not copy constructible, and similarly for `D&&` when `D` is not move constructible. This could be achieved by the traditional 'does not participate in overload resolution' wording, or similar.

Proposed change: Add a *Remarks:* clause to constrain the appropriate constructors.

[2017-02-28, Jonathan comments and provides concrete wording]

As well as addressing the NB comment, this attempts to make some further improvements to the current wording, which is a little strange. It incorrectly uses "`d`" to mean the constructor argument that initializes the parameter `d`, and unnecessarily explains how overload resolution works for lvalues and rvalues. It refers to the copy/move constructor of `D`, but the constructor that is selected to perform the initialization may not be a copy/move constructor (e.g. initializing a deleter object from an rvalue might use a copy constructor if there is no move constructor). The condition "`d` shall be reference compatible with one of the constructors" is bogus: reference compatible is a property of two types, not a value and a constructor, and again is trying to talk about the argument not the parameter.

Note that we could replace the "see below" in the signatures and paragraphs 9, 10 and 11 by declaring the constructors as:

```
unique_ptr(pointer p, const D& d) noexcept;  
unique_ptr(pointer p, remove_reference_t<D&& d>) noexcept;
```

I think this produces the same signatures in all cases. I haven't proposed that here, it could be changed separately if desired.

[Kona 2017-02-27]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

Modify [unique_ptr.single.ctor] paragraphs 9-11 as shown:

```
unique_ptr(pointer p, see below d1) noexcept;  
unique_ptr(pointer p, see below d2) noexcept;
```

-9- The signature of these constructors depends upon whether D is a reference type. If D is a non-reference type A , then the signatures are

```
unique_ptr(pointer p, const  $\mathsf{A}\&$  d) noexcept;  
unique_ptr(pointer p,  $\mathsf{A}\&\&$  d) noexcept;
```

-10- If D is an lvalue reference type $\mathsf{A}\&$, then the signatures are:

```
unique_ptr(pointer p,  $\mathsf{A}\&$  d) noexcept;  
unique_ptr(pointer p,  $\mathsf{A}\&\&$  d) = delete;
```

-11- If D is an lvalue reference type `const  $\mathsf{A}\&$` , then the signatures are:

```
unique_ptr(pointer p, const  $\mathsf{A}\&$  d) noexcept;  
unique_ptr(pointer p, const  $\mathsf{A}\&\&$  d) = delete;
```

Remove paragraph 12 entirely:

~~-12- Requires:~~

- ~~If D is not an lvalue reference type then~~
 - ~~If d is an lvalue or const rvalue then the first constructor of this pair will be selected. D shall satisfy the requirements of CopyConstructible (Table 24), and the copy constructor of D shall not throw an exception. This `unique_ptr` will hold a copy of d .~~
 - ~~Otherwise, d is a non-const rvalue and the second constructor of this pair will be selected. D shall satisfy the requirements of MoveConstructible (Table 23), and the move constructor of D shall not throw an exception. This `unique_ptr` will hold a value move constructed from d .~~
- ~~Otherwise D is an lvalue reference type. d shall be reference-compatible with one of the constructors. If d is an rvalue, it will bind to the second constructor of this pair and the program is ill-formed. [Note: The diagnostic could be implemented using a static_assert which assures that D is not a reference type. — end note] Else d is an lvalue and will bind to the first constructor of this pair. The type which D references need not be CopyConstructible nor MoveConstructible. This `unique_ptr` will hold a D which refers to the lvalue d . [Note: D may not be an rvalue reference type. — end note]~~

Modify paragraph 13 as shown:

-13- *Effects*: Constructs a `unique_ptr` object which owns p , initializing the stored pointer with p and initializing the deleter **as described above** `from std::forward<decltype(d)>(d)`.

Add a new paragraph after paragraph 14 (Postconditions):

-?- Remarks: These constructors shall not participate in overload resolution unless `is_constructible_v< $\mathsf{D}$ , decltype(d)>` is true.

2908. The less-than operator for shared pointers could do more

Section: 20.11.2.2.7 [util.smartptr.shared.cmp] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all other [issues](#) in [util.smartptr.shared.cmp].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 135

The less-than operator for shared pointers compares only those combinations that can form a composite pointer type. With the C++17 wording for the diamond functor, `less<>`, we should be able to support comparison of a wider range of shared pointers, such that `less<>::operator(shared_ptr< $\mathsf{A}$ >, shared_ptr< $\mathsf{B}$ >)` is consistent with `less<>::operator( $\mathsf{A}$  *,  $\mathsf{B}$  *)`.

Proposed change: Replace `less< $\mathsf{V}$ >` with just `less<>`, and drop the reference to composite pointer types.

[2017-03-02, Kona, STL comments and provides wording]

We talked about this in LWG, and I talked to Alisdair. The status quo is already maximally general (operator less-than in Core always forms a composite pointer type), but we can use diamond-less to significantly simplify the specification. (We can't use less-than directly, due to the total ordering issue, which diamond-less was recently "upgraded" to handle.)

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4640](#).

1. Change 20.11.2.2.7 [util.smartptr.shared.cmp] as depicted:

```
template<class T, class U>
bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

-2- *Returns:* `less<V>()(a.get(), b.get())`, where `V` is the composite pointer type (Clause 5) of `shared_ptr<T>::element_type*` and `shared_ptr<U>::element_type*`.

2911. An `is_aggregate` type trait is needed

Section: 20.15.4.3 [meta.unary.prop] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 143

An `is_aggregate` type trait is needed. The `emplace` idiom is now common throughout the library, but typically relies on direct non-list initialization, which does not work for aggregates. With a suitable type trait, we could extend direct non-list-initialization to perform aggregate-initialization on aggregate types.

Proposed change:

Add a new row to Table 38:

```
template <class T>
struct is_aggregate;
```

`T` is an aggregate type ([dcl.init.aggr])

`remove_all_extents_t<T>` shall be a complete type, an array type, or (possibly cv-qualified) void.

[2017-02-26, Daniel comments and completes wording]

The proposed wording is incomplete, since it doesn't update the `<type_traits>` header synopsis, also ensuring that the corresponding `is_aggregate_v` variable template is defined.

[2017-03-01, Daniel comments and adjusts wording]

There is only one minor change performed, namely to mark the constant in the `is_aggregate_v` variable template as `inline` following the LWG preferences for "full inline" bullet list (B2) from [P0607R0](#).

[2017-03-03, Kona, LEWG]

No interest in doing the `is-list-initializable` *instead*. No objection to getting that proposal also.

Want this for C++17?

SF | F | N | A | SA

2 | 8 | 1 | 3 | 0

Want this for C++20?

SF | F | N | A | SA

6 | 5 | 2 | 0 | 0

Remember to correct the wording to: "`remove_all_extents_t<T>` shall be a complete type or cv-void."

[2017-03-03, Daniel updates wording]

In sync with existing writing, this was changed to "`remove_all_extents_t<T>` shall be a complete type or cv void"

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

This will require a new compiler intrinsic; Alisdair checked with Core and they're OK with that

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.15.2 [meta.type.synop], <type_traits> header synopsis, as indicated:

```
// 20.15.4.3, type properties
[...]
```

```
template <class T> struct is_final;
template <class T> struct is_aggregate;

[...]
```

```
// 20.15.4.3, type properties
[...]
```

```
template <class T> constexpr bool is_final_v
    = is_final<T>::value;
template <class T> inline constexpr bool is_aggregate_v
    = is_aggregate<T>::value;

[...]
```

2. Add a new row to Table 42 — "Type property predicates":

```
template <class T> struct is_aggregate;

T is an aggregate type (8.6.1 [dcl.init.aggr])

remove all extents t<T> shall be a complete type or cv void.
```

[2921](#). packaged_task and type-erased allocators

Section: 30.6.10 [futures.task] **Status:** [Immediate](#) **Submitter:** United States **Opened:** 2017-02-03 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all other [issues](#) in [futures.task].

View all issues with [Immediate](#) status.

Discussion:

Addresses US 165

The constructor that type-erases an allocator has all of the problems of the similar function constructor that was removed for this CD. This constructor from packaged_task should similarly be removed as well. If we prefer to keep this constructor, the current wording is underspecified, as the Allocator argument is not required to be type satisfying the Allocator requirements, nor is allocator_traits used.

Proposed change:

Strike

```
template <class F, class Allocator>
packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

from the class definition in p2, and from **30.6.9.1 [futures.task.members]** p2. Strike the last sentence of 30.6.9.1p4. In p3, revise "These constructors" to "This constructor"

[Kona 2017-03-02]

Accepted as Immediate to resolve NB comment.

Proposed resolution:

This wording is relative to [N4618](#).

1. Modify 30.6.10 [futures.task] as follows:

Strike

```
template <class F, class Allocator>
packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

from the class definition in p2, and from [futures.task.members] p2.

2. Modify 30.6.10.1 [futures.task.members]/3:

Remarks: ~~These constructors~~ **This constructor** shall not participate in overload resolution if decay_t<F> is the same type as packaged_task<R(ArgTypes...)>.

3. Strike the last sentence of 30.6.10.1 [futures.task.members]/4:

The constructors that take an Allocator argument use it to allocate memory needed to store the internal data structures.

2934. optional<const T> doesn't compare with T

Section: 20.6.8 [optional.comp_with_t] **Status:** [Immediate](#) **Submitter:** Ville Voutilainen **Opened:** 2017-02-17 **Last modified:** 2017-03-03

Priority: Not Prioritized

View all issues with [Immediate](#) status.

Discussion:

Consider:

```
optional<const int> x = 42;
int y = 666;
x == y; // ill-formed
```

The comparison is ill-formed, because in [optional.comp_with_t]/1,

```
template <class T> constexpr bool operator==(const optional<T>& x, const T& v);
```

the T is deduced to be both `const int` and `int`, which is ill-formed.

Since it became apparent that the root cause of this issue is also causing difficulties with e.g. comparing `optional<string>` with literals etc., here's a p/r that

1. adds requirements for optional's comparisons with T
2. turns optional-optional comparisons into two-template-parameter templates, allowing comparing mixed optionals
3. turns optional- T comparisons into two-template-parameter templates, allowing comparing optionals with T and types comparable with T

[Kona 2017-02-28]

Accepted as Immediate to avoid having break ABI later.

Proposed resolution:

This wording is relative to [N4640](#).

1. Modify 20.6.2 [optional.syn], header <optional> synopsis, as indicated:

```
[...]

// 20.6.6 [optional.relops], relational operators
template <class T, class U>
constexpr bool operator==(const optional<T>&, const optional<U>&);
template <class T, class U>
constexpr bool operator!=(const optional<T>&, const optional<U>&);
template <class T, class U>
constexpr bool operator<(const optional<T>&, const optional<U>&);
template <class T, class U>
constexpr bool operator>(const optional<T>&, const optional<U>&);
template <class T, class U>
constexpr bool operator<=(const optional<T>&, const optional<U>&);
template <class T, class U>
constexpr bool operator>=(const optional<T>&, const optional<U>&);

[...]
```

```
// 20.6.8 [optional.comp_with_t], comparison with T
template <class T, class U> constexpr bool operator==(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator==(const U&, const optional<T>&);
template <class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator!=(const U&, const optional<T>&);
template <class T, class U> constexpr bool operator<(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator<(const U&, const optional<T>&);
template <class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator<=(const U&, const optional<T>&);
template <class T, class U> constexpr bool operator>(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator>(const U&, const optional<T>&);
template <class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
template <class T, class U> constexpr bool operator>=(const U&, const optional<T>&);
```

2. Modify 20.6.6 [optional.relops] as indicated:

```
template <class T, class U> constexpr bool operator==(const optional<T>& x, const optional<U>& y);
```

[...]

```

template <class T, class U> constexpr bool operator!=(const optional<T>& x, const optional<FU>& y);

[...]

template <class T, class U> constexpr bool operator<(const optional<T>& x, const optional<FU>& y);

[...]

template <class T, class U> constexpr bool operator>(const optional<T>& x, const optional<FU>& y);

[...]

template <class T, class U> constexpr bool operator<=(const optional<T>& x, const optional<FU>& y);

[...]

template <class T, class U> constexpr bool operator>=(const optional<T>& x, const optional<FU>& y);

[...]

```

3. Modify 20.6.8 [optional.comp_with_t] as indicated:

```

template <class T, class U> constexpr bool operator==(const optional<T>& x, const FU& v);

-?- Requires: The expression *x == v shall be well-formed and its result shall be convertible to bool. [Note: T need not be
EqualityComparable. — end note]

-1- Effects: Equivalent to: return bool(x) ? *x == v : false;

template <class T, class U> constexpr bool operator==(const FU& v, const optional<T>& x);

-?- Requires: The expression v == *x shall be well-formed and its result shall be convertible to bool.

-2- Effects: Equivalent to: return bool(x) ? v == *x : false;

template <class T, class U> constexpr bool operator!=(const optional<T>& x, const FU& v);

-?- Requires: The expression *x != v shall be well-formed and its result shall be convertible to bool.

-3- Effects: Equivalent to: return bool(x) ? *x != v : true;

template <class T, class U> constexpr bool operator!=(const FU& v, const optional<T>& x);

-?- Requires: The expression v != *x shall be well-formed and its result shall be convertible to bool.

-4- Effects: Equivalent to: return bool(x) ? v != *x : true;

template <class T, class U> constexpr bool operator<(const optional<T>& x, const FU& v);

-?- Requires: The expression *x < v shall be well-formed and its result shall be convertible to bool.

-5- Effects: Equivalent to: return bool(x) ? *x < v : true;

template <class T, class U> constexpr bool operator<(const FU& v, const optional<T>& x);

-?- Requires: The expression v < *x shall be well-formed and its result shall be convertible to bool.

-6- Effects: Equivalent to: return bool(x) ? v < *x : false;

template <class T, class U> constexpr bool operator<=(const optional<T>& x, const FU& v);

-?- Requires: The expression *x <= v shall be well-formed and its result shall be convertible to bool.

-7- Effects: Equivalent to: return bool(x) ? *x <= v : true;

template <class T, class U> constexpr bool operator<=(const FU& v, const optional<T>& x);

-?- Requires: The expression v <= *x shall be well-formed and its result shall be convertible to bool.

-8- Effects: Equivalent to: return bool(x) ? v <= *x : false;

template <class T, class U> constexpr bool operator>(const optional<T>& x, const FU& v);

-?- Requires: The expression *x > v shall be well-formed and its result shall be convertible to bool.

-9- Effects: Equivalent to: return bool(x) ? *x > v : false;

template <class T, class U> constexpr bool operator>(const FU& v, const optional<T>& x);

-?- Requires: The expression v > *x shall be well-formed and its result shall be convertible to bool.

-10- Effects: Equivalent to: return bool(x) ? v > *x : true;

template <class T, class U> constexpr bool operator>=(const optional<T>& x, const FU& v);

```


-?- Requires: The expression `*x >= v` shall be well-formed and its result shall be convertible to `bool`.

-11- *Effects*: Equivalent to: `return bool(x) ? *x >= v : false;`

template <class T, class U> constexpr bool operator>=(const ~~T~~U& v, const optional<T>& x);

-?- Requires: The expression `v >= *x` shall be well-formed and its result shall be convertible to `bool`.

-12- *Effects*: Equivalent to: `return bool(x) ? v >= *x : true;`